

FMCG Multi-Country Sales Dataset Analysis

(Validation & Preparation)

Document no.: 03

Pages: 03

Date: 4 September, 2026

3.1 Import Dataset in MySQL

As we've performed initial data quality check and dataset cleanup in Power Query editor of Excel, now it is time to import the cleaned dataset into MySQL to handle heavy loads and further analysis. Here, step by step process is mentioned chronologically (I'm using MySQL Workbench 8.0 platform)–

1. Open MySQL Workbench and create a fresh SQL query tab. Give a name to that query tab (Dataset_import.sql). Then create a database (I've named it 'fmcg_recordbook') and select it for current session.
2. To import dataset into our created database from local drive, we need to do some settings first. Go to Database tab > Manage Connections > select the connection from the left pane > click 'Advanced' > in 'Others' space, paste this 'OPT_LOCAL_INFILE=1' and close the window. Then in the SQL query tab, write 'SET GLOBAL local_infile=1;' and execute. Next to prevent connection loss of MySQL query process with such large dataset, go to Edit tab > Preferences > select 'SQL Editor' from the left pane > MySQL session > DBMS connection read timeout interval (in seconds) > make it 300 or more instead of 30 sec.
3. Create a table that is going to hold the entire dataset. Now to load such huge amount of data (1.1M rows, 200MB size) using regular method will lead to freeze or crash the Workbench background process. So to safely create a table, right click the database name on the 'Schemas' pane and select 'Table Data Import Wizard'. Then select the dataset file path, give a name of the table (sales_record) and click next. A table will start to from and import data from the dataset. After 2/3 sec. stop immediately! Cause this process will take hours to complete. We are going to use a much faster method.
4. Our table structure is created within that 2/3 sec. time. Now use TRUNCATE command in the sql tab to erase any left-over data. Now the table should have all 33 columns but zero rows. Now we gonna use LOAD DATA LOCAL INFILE... command. This will import all 1.1M rows into the table within 1-2 minutes!

3.2 Verify Import

To check the dataset has successfully imported into the table (sales_record) without any issue, we will do some inspections –

1. Use 'DESCRIBE sales_record;' command to see all 33 columns with their data types [In my observation, all fields were in correct data types except 'sales_date' field, which was in TEXT type rather than DATE type. So, I fixed it by ALTER TABLE ... CHANGE COLUMN ... command]
2. Use SELECT * FROM sales_record LIMIT 10; command that will show first 10 records of entire dataset.
3. Use SELECT COUNT (*) FROM sales_record; it will count total records/rows in the table. It should be exactly 1100000 rows.
4. To see the timeline of the records (starting date & ending date), use SELECT MIN (sales_date) AS first_date, MAX (sales_date) AS last_date FROM sales_record; this should be 1-Jan-21 and 31-Dec-23.
5. To check the last time that any suspected field might have a null or not, use SELECT (column_name IS NULL) AS null_column_name FROM sales_record; This will result to 0 if there is no null.

3.3 Establish a Primary Key

In the understanding phase of the dataset, I've noticed that there is not a single field that contains a unique id for each record – 1.1M unique id for 1.1M records. Absence of such uniqueness makes it hard to pull of data of specific records during analytical stage. That's why we need to establish uniqueness to define each and every records individually. Here I've used another SQL file named 'Validation'.

To do that (make primary key), I looked upon a combination of fields that can make each record unique. I've tried combinations of: date+store+sku & date+store+sku+channel. Both of them returned exact same number of uniqueness (1100000 unique records) from this approach:

```
SELECT
COUNT (*) AS total_rows, COUNT (DISTINCT CONCAT (sales_date, '|', store_id, '|', sku_id))
AS distinct_date_store_sku
FROM sales_record;
```

As primary key is formed using the least number of fields, I used date+store+sku combination rather than date+store+sku+channel. But what does this mean? This combination means, for a particular date and in a particular store, a particular item sold aggregately on that day is a record that is unique that is very unlikely to match with any other stores on that specific day.

Now the grain is found, it's time to allocate a unique id for each grain (primary setups). So, I added another column (sales_row_id) that is AUTO INCREMENTED to assign unique number for each record. Now total columns are 34.

3.4 Prepare a Secondary Table for Analysis

To preserve original cleaned data records (sales_record table) from altering/tampering while we perform analysis, I've created another table (sales_analysis) and copied the data and structure from sales_record like as it is. To do this, I used this sql command:

```
CREATE TABLE sales_analysis AS
SELECT * FROM sales_record WHERE 1=0;
```

mathematically, 1=0 is a FALSE statement, so it only copies the structure and data types, but not a single row of info in the new table.

```
ALTER TABLE sales_analysis
ADD COLUMN sales_row_id INT AUTO INCREMENT FIRST, ADD PRIMARY KEY (sales_row_id);
```

This adds a new column 'sales_row_id' at the first of the record book and assigns it as primary key field.

```
INSERT INTO sales_analysis SELECT NULL, s.* FROM sales_record s;
```

Now this command copies the entire 'sales_record' table data to copy into 'sales_analysis' table. And the NULL triggers the AUTO INCREMENT command to start filling each row of 'sales_row_id' with unique number at incremental rate.

Now that a new table is formed, its time to set index numbers on the fields that are going to be frequently used/called upon in analytics stage (fields like sales_date, sku_id, store_id, country etc.) by such query command:

```
CREATE INDEX idx_sales_date ON sales_analysis (sales_date);
```

```
CREATE INDEX idx_sku_id ON sales_analysis (sku_id (50));  
CREATE INDEX idx_store_id ON sales_analysis (store_id (50));  
CREATE INDEX idx_country ON sales_analysis (country (50));  
CREATE INDEX idx_category ON sales_analysis (category (50));  
CREATE INDEX idx_channel ON sales_analysis (channel (50));  
CREATE INDEX idx_promo_flag ON sales_analysis (promo_flag);
```

Indexing makes actions like filtering, grouping much more convenient and memory efficient, which means faster process time.

[Note: fields that have TEXT data type requires explicit declaration of how many characters allowed to be indexed. For example, 'country' field is a TEXT type field so it requires a length to declare for indexing, which I set is 50]

N4H1Y4N-59